IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received April 5, 2021, accepted April 21, 2021, date of publication April 29, 2021, date of current version May 18, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3076599

DESOLATER: Deep Reinforcement Learning-Based Resource Allocation and Moving Target Defense Deployment Framework

SEUNGHYUN YOON[1], (Graduate Student Member, IEEE), JIN-HEE CHO[2], (Senior Member, IEEE), DONG SEONG KIM[3], (Senior Member, IEEE), TERRENCE J. MOORE[4], (Member, IEEE), FREDERICA FREE-NELSON[4], AND HYUK LIM[5], (Member, IEEE)

[1]School of Electrical Engineering and Computer Science, Gwangju Institute of Science and Technology, Gwangju 61005, Republic of Korea
[2]Department of Computer Science, Virginia Tech, Falls Church, VA 22043, USA
[3]School of Information Technology and Electrical Engineering, The University of Queensland, St Lucia, QLD 4067, Australia
[4]U.S. Army Research Laboratory, Adelphi, MD 20783, USA
[5]AI Graduate School, Gwangju Institute of Science and Technology (GIST), Gwangju 61005, Republic of Korea

Corresponding author: Hyuk Lim (hlim@gist.ac.kr)

This work was supported in part by the U.S. Army Combat Capabilities Development Command (CCDC) International Technology Center-Pacific (ITC-PAC), and in part by the CCDC Army Research Laboratory (CCDC-ARL) under Grant FA5209-19-P-A056.

ABSTRACT The recent development of autonomous driving technologies has led to the proliferation of research on sensors and electronic equipment inside a vehicle. To deal with security concerns of in-vehicle networks, various deep learning (DL) and reinforcement learning (RL) have been developed to enhance in-vehicle security. However, the DL/RL agents are vulnerable to adversarial perturbation, where an attacker can perform a manipulation attack to interfere with the agent's operation. In this work, we aim to develop two key mechanisms to build secure in-vehicle networks: (1) RL-based proactive defense mechanism to achieve multiple objectives of minimizing system security vulnerabilities while maximizing service availability; and (2) a resilient RL method that allows an agent to operate in the presence of adversarial disturbances that neutralize the system security. To this end, we propose, DESOLATER (**D**rl-based r**ESO**urce a**L**location **A**nd m**T**d d**E**ployment f**R**amework), which is a multi-agent deep reinforcement learning (mDRL)-based network slicing technique that can help determine two key network management decisions: (1) link bandwidth allocation to meet quality-of-service (QoS) requirements; and (2) the frequency of triggering IP shuffling as a proactive defense mechanism not to hinder service availability by maintaining normal system operations. We also introduce an anomaly detection mechanism with a memory-based RL technique to enhance the resiliency of the RL agents in a partially observable environment under the situation that adversarial attackers manipulating observation information. Through extensive simulation experiments, we validate that the proposed robust mDRL algorithm can help the deployed proactive security mechanism achieve both security and network performance improvement in the presence of adversarial attacks.

INDEX TERMS Deep reinforcement learning, in-vehicle network, moving target defense, network slicing, partial observability, software-defined networking.

I. INTRODUCTION

Along with significant developments in autonomous driving, vehicle-to-everything (V2X) communications have been adopted to further enhance tactical vehicular technologies. Accordingly, the significant evolution of vehicular technologies has led to the increased need of in-vehicle cybersecurity, such as securing an in-vehicle network consisting of several sensors, actuators, and electric control units (ECUs). However, state-of-the-art in-vehicle networking technologies have shown the following limitations. First, the current legacy in-vehicle networking protocols suffer from a lack of scalability and limited bandwidth. Since an in-vehicle network uses complex and inflexible heterogeneous protocols, such

The associate editor coordinating the review of this manuscript and approving it for publication was Shaohua Wan.

as control area network (CAN), local interconnect network (LIN), and FlexRay, each vehicle component should be connected to dedicated protocols according to their communication requirements, which are bandwidth-constrained and not scalable. Second, the existing in-vehicle protocols lack security features. This is because common security requirements (e.g., integrity or confidentiality (with encryption)) require high computational power or sufficient bandwidth for rounds of communications associated with different phases of key management, including key issuance, distribution, update, and revocation. In this work, we aim to tackle these two challenges by developing an autonomous decision making framework that provides solutions for effective bandwidth allocation by leveraging the network slicing technology and efficient implementation of a proactive defense mechanism, called *moving target defense (MTD)* [1].

In particular, in order to introduce a security mechanism to proactively protect an in-vehicle network, we adopt MTD techniques that can dynamically change attack surfaces to increase uncertainty or confusion for the attacker to make poor attack decisions. Among MTD techniques, we choose to adopt a network shuffling-based MTD technique (e.g., IP/port/MAC address mutations [1]) with the aim of changing the attack surface (e.g., network addresses) and invalidating any collected intelligence by the attacker performing reconnaissance attacks. Hence, this will effectively hinder the attacker to launch its attack or prevent it from escalating the attack to the next level. The key design challenge in deploying shuffling-based MTD is how to determine when to trigger MTD operations, which will generate additional overhead while minimizing security vulnerabilities of a system. This additional overhead may often introduce a potential performance degradation of a system, such as a system not responding to users in a timely manner.

In this work, we aim to develop a deep reinforcement learning (DRL)-based resource allocation and MTD deployment framework, the so-called DESOLATER (Drl-based rESOurce aLlocation And mTd dEployment fRamework). The DESOLATER aims to effectively identify an optimal amount of link bandwidth to each network slice which has a DRL agent (i.e., network slice controller) running and an optimal interval to trigger IP shuffling-based MTD based on the critical tradeoff between security and performance, as discussed above. To be specific, the proposed DESOLATER will achieve the following objectives:

1) Identify an optimal amount of link bandwidth (i.e., network resources) allocated to each network slide to minimize a packet loss rate.
2) Identify an optimal 'triggering' interval for IP shuffling-based MTD with the aim of minimizing security vulnerability (i.e., higher vulnerability when data are sent to the same IP address for a longer period) while minimizing a packet loss rate.
3) Enhance the stability of learning when the proposed multi-agent DRL (mDRL) is deployed in the presence of adversarial attacks.

By leveraging the merits of SDN and network slicing technology, network resources are managed in a centralized way to provide an agile configuration capability, and in-vehicle network components are protected securely by MTD functionalities. Our main contributions are summarized as follows:

- We proposed a unified SDN-based in-vehicle network architecture which supports various requirements of legacy in-vehicle networking protocols.
- We proposed an SDN-based network slicing technique for constructing isolated logical virtual in-vehicular networks for each type of traffic flows with different QoS and security requirements.
- We presented real-time MTD-ready functionalities designed for critical traffic flows between sensors/actuators and controller units inside and outside of the in-vehicle network with delay and reliability constraints.
- We proposed a robust multi-agent deep reinforcement learning (mDRL) algorithm called multi-agent recurrent deterministic policy gradient with anomaly detector (MARDPG-AG) that enables the agents to learn robustly even in the presence of partial or manipulated observations in system states of a given environment.
- We implemented the proposed MTD technique in the in-vehicular SDN testbed and demonstrated its outperformance in terms of the resource allocation success rate and security vulnerability.

The rest of this paper is organized as follows. Section II provides the overview of related work and background technologies leveraged in this work. Section IV presents our network model, node model, and attack model. Section III provides an overview of the related literature on MTD and DRL-based resource allocation. Section V describes the details of the proposed approach in terms of formulating key parameters in mDRL, anomaly detector to filter manipulated observations, partially observable environment, the formulation of multi-agent RDPG. Section VI demonstrates the experimental results and discusses their overall trends observed along with in-depth analysis. Section VIII summarizes the key findings and concludes the paper with future work directions.

II. BACKGROUND

In this section, we provide background knowledge on the existing legacy in-vehicle networking protocols. Then, we provide backgrounds of legacy in-vehicle networking protocols, SDN/NFV, SDN-based address shuffling, and deep reinforcement learning techniques for better readability.

A. LEGACY IN-VEHICLE NETWORK PROTOCOLS

In-vehicle network protocols such as control area network (CAN), local interconnect network (LIN), media oriented systems transport (MOST), and FlexRay have been used to interconnect the in-vehicle components. All of these communication buses have been specifically developed for the

TABLE 1. Current in-vehicle networking protocols.

Protocol	Maximum data rate	Topology	Security	Target domain	Description	Latency requirement	Bandwidth requirement
CAN CAN FD	1 Mbps 16 Mbps	Bus, Star, Ring	Not supported	Low bandwidth real-time control, safety applications	Powertrain and Chassis control	< 10 μs	Low
LIN	19.2 Kbps	Bus	Partially	Low-performance modules	Light control, colling fan control, actuator/sensor control	< 100 ms	Low
MOST	150 Mbps	Ring	Partially	Infotainment system, human-machine interface	Vehicle display control, infotainment, multimedia transmission	< 150 ms	High
FlexRay	20 Mbps	Bus, Star	Partially	Real-time control, safety/critical applications	Safety data (vedio, audio), steering, breaking control	< 50 ms	Medium
Ethernet	100 Mbps	Bus, Star	Partially	Infotainment system, in-vehicle diagnostics	Multimedia transmission	< 150 ms	High

in-vehicle environment. We briefly describe representative protocols as below:

- Controller Area Network (**CAN**) is an automotive-specific networking protocol, and typically used to transmit control traffic between ECUs within the vehicle. The maximum bus speed of CAN is 1 Mb/s at lengths of up to 40m. CAN is mainly used in body and engine control. The devices that are connected through a CAN network are typically vehicle control devices, sensors, and actuators.
- Local Interconnect Network (**LIN**) was developed for low bandwidth small serial control message transmission such as door lock control, light sensor, and steering wheel control with low cost. LIN is a broadcast serial network and all messages are initiated by the master node with at most one slave node replying to a given message identifier.
- Media Oriented Systems Transport (**MOST**) is a technology for high-speed multimedia transmission including audio, video, and voice. As the rising number of infotainment applications and video applications, MOST 150 supports up to 150 Mbps using optical cable.
- **FlexRay** provides error tolerance communications and deterministic data transmission. It is designed to be faster and more reliable than CAN, so that FlexRay is used for safety and critical applications in the vehicle such as steering and braking control.
- **Ethernet**-based communication technology has recently emerged in the in-vehicle network domain [2] to mitigate the adverse effect of the legacy in-vehicle protocols. However, the use of Ethernet-based technology is currently only applied to ECU diagnosis via the Internet.

Table. 1 lists the target domain description and requirements and characteristics of general in-vehicle networking protocols.

B. SOFTWARE-DEFINED NETWORKING (SDN)

The SDN technology decouples a network control plane from a data-forwarding plane to improve flexibility, robustness, and/or programmability to a networked system. In conventional networks, a routing decision (i.e., packet forwarding) is made at each switch, which often causes lack of controls over switches, leading to running a system in a less optimized manner. In an SDN environment, an SDN controller takes control and can effectively run all packet forwarding operations in a centralized manner. Owing to this flexibility and programmability, the use of SDN technology becomes popular in developing various network management applications and cybersecurity applications in networked systems. For example, the SDN controller allows a switch to rewrite packet headers and steer traffic toward a specific network host by simply updating the forwarding table of the switch via an OpenFlow (OF) protocol [3]. In [2], the authors proposed an SDN-based in-vehicle network that enables in-vehicle data sources interoperability that allows various ECU sources to share the same SDN environment.

Network function virtualization (NFV) decouples network functions, such as firewall, domain name system (DNS), and intrusion detection system (IDS) from the physical hardware by using virtualization technology [4]. By combining the SDN and the NFV technologies, network traffic control and security have been enhanced by arranging virtualized network functions, such as virtualized load balancers, firewalls, and IDSs at a suitable time and place [5] with the use of service function chaining (SFC) technology. SFC facilitates the differentiated traffic forwarding policies and security policies and helps network traffic efficiently traverse through a set of network services or functions.

C. PRINCIPLE OF SDN-BASED ADDRESS SHUFFLING

In SDN-based address shuffling mechanisms, network node's real addresses (i.e., physical addresses) are unchanged, and virtual addresses of a node such as virtual IP/MAC addresses are shuffled periodically by the SDN controller. Virtual addresses of a node are the addresses used to forward data packets to the destination node, and the SDN controller updates the flow tables of SDN switches in the network based on its real-virtual address mapping table. As shown in Figure 1, an MTD controller in the SDN controller updates flow tables of the SDN switches in the network to convert real IP address information to virtual IP address and vice versa. In this case, all real IP address information in the data packet is converted into virtual addresses and hidden during packet transmission. The flow rules reconvert only the virtual address of the destination node in a packet header to its real address at the switch right before a packet is delivered to the destination host. Therefore, this address shuffling process ensures normal communication, and only the SDN controller

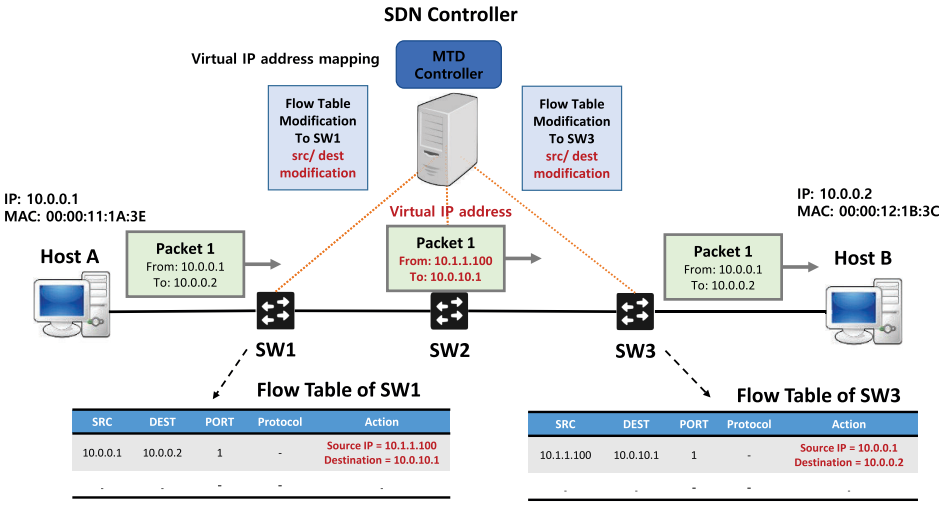


FIGURE 1. SDN-based address shuffling mechanism.

knows the real addresses of both the source node and the destination node.

D. REINFORCEMENT LEARNING (RL)

In RL, an agent selects their next action based on a policy π, which is a probability distribution over actions in the states, where $\pi(s, a)$ denotes the probability of taking action a in state s. In most real-world applications, the action and state spaces are continuous, so we use a function approximator to scale-down the problem, which is parameterized by θ. Typically, in this work, we use a deep neural network (DNN) as the function approximator and utilize a policy learning algorithm called *policy gradient method* to learn an optimal policy directly using a gradient descent method. The objective of learning is to maximize expected cumulative discounted reward, and the gradient of the objective function $J(\theta)$ is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(s, a) Q^{\pi_\theta}(s, a)], \tag{1}$$

where $Q^{\pi_\theta}(s, a)$ represents the expected cumulative discounted reward from selecting action a from the state s and then following the policy π_θ.

These policy-based methods are usually more efficient to handle the continuous world problem than value-based methods because the computation complexity of value-based approaches (i.e., maintaining state-value table and/or action value table) is too high when there are an infinite number of actions and states. In order to solve the curse of dimensionality in value function-based learning, we directly parametrize the policy by θ, and it is denoted by $\pi_\theta(s, a)$. As the learning progresses, the policy parameters are subsequently updated using gradient ascent using the following equation:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(s_t, a_t) v_t, \tag{2}$$

where v_t represents episodic reward at time t, and α is a gradient step size. By using this method, we can update θ in the direction of the policy gradient, $\nabla J(\theta)$, to find θ in the policy π_θ that will get the maximum return. Recently, to reduce non-stationarity due to high variance gradient estimates in stochastic policy gradient methods, deterministic policy gradient (DPG) [6] and deep deterministic policy gradient (DDPG) [7] have been proposed. DDPG adopts a model-free, actor-critic algorithm to successfully deal with large-scale continuous action space problems. In DDPG, two neural networks are maintained to learn the policy (i.e., actor network) and value function (i.e., critic network) by an agent separately. DDPG is well-known to perform well in continuous action spaces and MDP environments.

Taking one step further, a multi-agent system comes into play along with DRL. This is called *multi-agent DRL*, namely mDRL. mDRL has been proposed to apply DRL for multi-agent settings. To resolve the curse of high dimensionality, efforts to improve scalability have been made by introducing observation embedding to be mapped to features and parameter noise-based perturbation for encouraging exploration [8]. The application examples using DRL include a deep-Q-network based DRL to solve multi-workflows scheduling problem in cloud computing [9] or path planning of multi-agent constrained formation using a parallel deep Q-network [10].

In mDRL, multiple agents collaborate with each other by sharing their information for maximizing the long-term reward with their best choice of actions. The concept of the so-called 'centralized learning and distributed execution' is adopted as a design principle to achieve DRL agents' goal [11]. Recently, to reduce non-stationarity due to high variance gradient estimates in policy gradient methods and dynamics of multi-agents' decision making, multi-agent deep deterministic policy gradient (MADDPG) algorithm [12] has

been proposed. We refined an actor-critic network that considers a multi-agent system where agents exchange information (i.e., observations, rewards, and actions) in order to mitigate non-stationarity in identifying a policy.

III. RELATED WORK

A. MTD ON VARIOUS DOMAINS

MTD has been introduced as an effort to proactively thwart potential attackers as an intrusion prevention mechanism. The key underlying idea of MTD is to increase uncertainty and confusion for attackers by changing attack surfaces (i.e., system or network configurations) with the aim of invalidating intelligence collected by the attackers. Nowadays, the functionalities of network programmability of SDN, such as packet header modification, dynamic address management, and rerouting, can enable the execution of MTD operations to further enhance cybersecurity that provides proactive intrusion prevention [13]. In [14], the authors developed an SDN-based topology deception system that virtually creates a network topology using honeypots. In [15], an optimal network reconfiguration method is proposed by applying network topology shuffling-based MTD in SDNs. The authors showed that overall network security with the MTD is increased when the network routing paths are diversified by shuffling. In [16], an OF-based random host mutation (OF-RHM) architecture in SDNs is proposed in which an SDN controller manages the mapping of a real IP to a virtual IP. In [17], the authors proposed a scalable port-hopping MTD using vulnerability scores to generate a scalable attack graph in an SDN environment. In [18], the authors devised a port hopping-based MTD in an SDN environment to deal with operating system (OS) fingerprinting and network reconnaissance attacks.

Although MTD has been applied in various forms under different systems or network environments [19]–[21], MTD techniques complying with the given characteristics of vehicular networks have not been well explored, particularly for in-vehicle networks. There have been a few MTD techniques developed for vehicular networks [19], [22], mobile ad-hoc networks (MANET) [23], and CAN environment [24]. However, no prior SDN-based MTD techniques have been developed to protect the in-vehicle SDN environment, which is studied in our work.

B. DRL-BASED DYNAMIC RESOURCE ALLOCATION

DRL has been widely used as one of the effective methods of maximizing long-term rewards for dynamic resource allocation methods. DRL algorithms have been successfully applied as an alternative to solving problems that were hard to deal with due to the dynamic complexity associated with resource allocation in traditional optimization algorithms [25]–[28]. In [26], the authors proposed a DRL-based resource scheduling method to enable dynamic adjustment of network resources to meet traffic demand. In [25], a DQN-based network resource provisioning scheme was proposed to increase network resource utilization and QoS satisfaction in a dynamic wireless network environment. The proposed resource provisioning scheme dynamically adjusts the network resource allocated to slices to meet different QoS characteristics of network traffics. In [27], the authors proposed an autonomous slice resource allocation scheme using the DRL approach in virtualized radio access networks (RANs). The DRL agent (i.e., especially DQN algorithm) learns an optimal resource provisioning strategy for each network slice to enhance the QoS satisfaction. In [29], [30], the authors proposed a multi-agent DRL based network and security resource allocation in an in-vehicle SDN environment. The proposed mDRL-based resource allocation method decides how much bandwidth should be allocated and how often MTD should be performed in the slice. Although DRL methods have been applied in various network resource management problems and network slicing techniques, however, no research has been conducted in adversarial environments with attackers targeting the DRL agents, which is studied in our work.

IV. SYSTEM MODEL

In this work, we leverage mDRL to develop a network slicing technique, aiming to solve a network and security resource allocation problem. We consider slicing a network in a logical sense in order to maximize effective utilization of the network and security resources. There are N network slices and N slice SDN controllers in an in-vehicle SDN. Each slice is controlled by its own slice controller, and the slice controller is considered as a DRL agent to take an optimal action to maximize its long-term return, which is derived by a function of system security (for minimizing security vulnerability) and networking performance (for maximizing service availability).

Due to the limitations of heterogeneous legacy in-vehicle networking protocols in terms of network bandwidth and inflexibility of the conventional protocols (e.g., network size), SDN technology recently emerged in the in-vehicle network domain [29], [31]. By using the SDN functionalities, we can integrate multiple heterogeneous data sources into a unified SDN environment. Inter-networking adaptors in each in-vehicle component convert conventional data sources to IP packets and vice versa. This allows the SDN controller to manage all information of the in-vehicle environment regardless of the legacy protocols.

A. IN-VEHICLE NETWORK MODEL

In Fig. 2, we present an SDN-based in-vehicle network architecture that can integrate multiple heterogeneous data sources into a unified SDN environment. In order to interconnect in-vehicle ECUs through an SDN, we consider a *universal inter-networking adaptor* that performs the conversion of heterogeneous data sources to IP packets and vice versa. The adaptor allows the various data sources to share the information in the SDN environment regardless of the legacy protocols, such as CAN, LIN, and MOST.

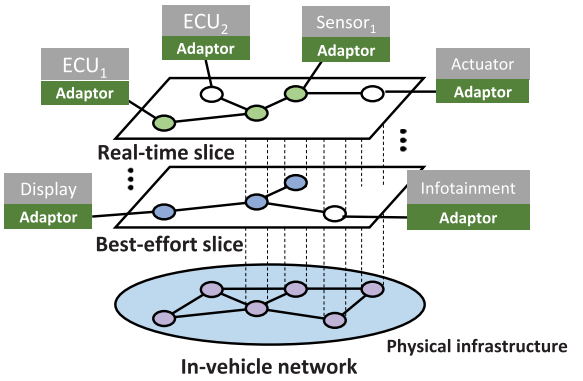


FIGURE 2. In-vehicle SDN environment.

Network slicing logically partitions a single network infrastructure into multiple logical slices. Each logically partitioned slice represents an independent virtualized end-to-end network environment to meet the specific needs of a service. The network resources, such as communication resources (e.g., throughput, bandwidth, switch queue rate) and computational resources (e.g., computing power, network functions), can be flexibly configured by Quality of Service (QoS) demands of each network slice [32], [33]. With the help of the SDN and NFV technology, an in-vehicle network can be logically partitioned into several network slices, where each slice is configured and provisioned for the particular in-vehicle service according to their characteristics and requirements. This edge-side in-vehicle network slicing technique is an essential technology for building a unified in-vehicle network while meeting the QoS requirements of various vehicle components. For example, in our proposed in-vehicle network architecture, tightly associated ECUs, such as engine control ECUs and stability control ECUs, are mapped to the same network slice while other nodes that are not mapped to the slice are logically isolated or separated from these nodes belonging to the network slice.

1) USE OF NETWORK SLICES IN MDRL-BASED RESOURCE ALLOCATION AND SECURITY OPTIMIZATION

Assume that there are N network slices operating in a given in-vehicle network, where all the slices share the in-vehicle network infrastructure resources. Each network slice is controlled by a slice SDN controller which performs the function of forwarding data packets and applying MTD security policies in the slice. Those N slice controllers are controlled by one central orchestrator. Note that a slice controller can monitor the network status of each slice and can measure the security (e.g., a number of IDS alarms, scanning success rate) of the components. The central orchestrator determines the resource allocation for each slice. We consider the cooperative decentralized mDRL approach, meaning that each slice controller has their DRL model for the actions and observations of all the agents (i.e., the slice SDN controllers). Each slice SDN controller (i.e., an agent) firstly selects the action space, including: (1) appropriate network resources (i.e., link bandwidth) for traffic forwarding in the slice; and (2) suitable security resource (i.e., an IP address shuffling interval) for the network security for the slice. The observation space of the proposed method includes: (1) QoS of the components in each slice; (2) current network flow information; (3) security vulnerability (i.e., a probability that an attacker will successfully scan/identify a target host) of the slice components; and (4) additional overhead (e.g., a packet drop rate) caused by the IP shuffling-based MTD. Lastly, each slice SDN controller receives a reward estimated based on the degree of changed QoS, system security, and defense cost.

2) LINK BANDWIDTH ALLOCATION FOR QoS

We consider a link bandwidth allocation problem to maximize the performance (i.e., service availability) of the slice components. In-vehicle network traffic changes dynamically based on the communication status between the external vehicles or infrastructures and the communication between internal components. By dynamically allocating bandwidth resources according to the traffic state inside the slice, it is possible to use the limited resources of the in-vehicle infrastructure with maximum resource utilization.

3) IP SHUFFLING CAPABILITY FOR SECURITY

We apply a virtual IP (vIP) shuffling-based MTD technique to the in-vehicle SDN, where each node's vIP address is periodically mutated by the slice controller (i.e., slice SDN controller). When a vIP packet from an ECU node comes to the SDN switch, the slice controller overwrites the IP address including the CAN ID information with a random virtual address, and then the virtual address is used for forwarding the packets. The newly generated virtual address should not overlap the recently used addresses. The source IP address is shuffled periodically and the original IP address containing the CAN ID information of an ECU node is only stored in the SDN controller. Thus, an attacker (i.e. eavesdropper) cannot obtain the real IP address of the packet, and accordingly, the collected information is invalidated after the MTD shuffling process. By applying our mDRL-based network slicing technique, we can apply the proper shuffling interval for each slice without degrading the QoS. This will lead to scalable, affordable, proactive security without high service interruptions.

B. ATTACK MODEL

DRL is known to be vulnerable to adversarial perturbations to their observations, similar to adversarial examples for DL-based classifiers. We assume that an adversary has the ability to access automotive diagnostic tools or sensors to acquire the information of an in-vehicle network by eavesdropping data packets in the network. The eavesdropped information provides the attacker with information about the role of sensors and ECUs in the in-vehicle network. Moreover, the attacker can inject malicious data packets into the network based on these previously eavesdropped packets. We

TABLE 2. Description of notations.

Parameter	Meaning
t	Time step
N	The number of slices in the network
o_i^t	The maximum amount of traffic in the slice i at t
l_i^t	The total packet loss for every in-vehicle components in the slice i at t
d_i^t	The average packet drop rate caused by address shuffling in the slice i at t
v_i^t	The average security vulnerability degree in the slice i at t
r_i^g	The reward of the satisfaction rate of a slice i
b_i^t	The allocation bandwidth resource at t
$\hat{b}_i^t$	The maximum data rate in the slice at t
P_n^t	the total number of packets sent by the component n at time t
$\hat{P}_n^t$	The maximum number of packets sent by the component n with the same IP address at time t
$\mathbf{h}$	The entire history of all agents

consider the state information manipulation attack as shown in Fig. 3, in which an attacker sends false state information of a slice to the DRL agent (i.e., a slice controller) to invalidate the agent's DRL-based resource allocation process. For example, an agent that receives fake information from the attacker tries to get more bandwidth resources according to the manipulated state. Then, other agents (i.e., slices) may not obtain the necessary resources because of the agent under the manipulation attack. As a result, all agents cannot guarantee QoS of their slice and decrease MTD shuffling interval, so the proper level of security cannot be maintained in the system. In order to deal with this type of attack, we consider an anomaly detector that can detect the state manipulation attacks in our proposed DESOLATER framework, described in the following section (see Section V-B).

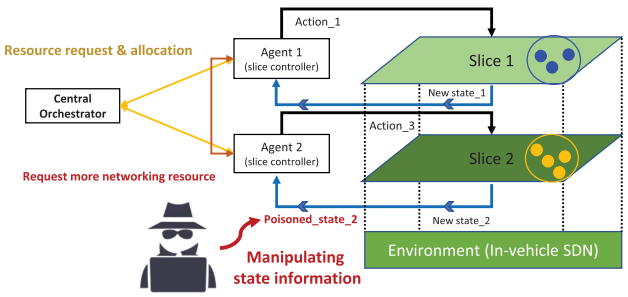


FIGURE 3. An example scenario of a state manipulation by adversarial attacks.

V. PROPOSED DESOLATER FRAMEWORK

In this section, we describe our proposed DESOLATER framework, which aims to identify optimal settings of resource allocation and system security in the presence of information manipulation attacks. This section provides the detailed formulation of mDRL in state and action spaces and a reward function, the proposed anomaly detector to detect manipulated observations, modeling of partial observation, and parameterization of our proposed multi-agent RDPG algorithm.

A. FORMULATION OF STATE SPACE, ACTION SPACE, AND REWARD SPACE

We use a Markov decision process (MDP) to formulate an RL-based optimization problem. We formulate a state, an action, and a reward function in order for a DRL agent to identify an optimal action to maximize the accumulated reward as follows.

1) STATE SPACE

The state space of the network slicing for IP shuffling-based MTD scheme consists of the current network traffic state and QoS state of the components in a network slice, an additional overhead state, and a security vulnerability state. The set of the maximum amount of traffic in the slice at time step t is denoted as $\mathcal{O} = \{o_1^t, o_2^t, \ldots, o_i^t \ldots, o_N^t\}$, where N is the number of slices in the network. We defined the set of QoS state of the components in the slice at time step t as $\mathcal{Q} = \{l_1^t, l_2^t, \ldots, l_i^t \ldots, l_N^t\}$, where l_i^t represents the total packet loss for every in-vehicle components in the slice i at time t. The set of additional overhead of the slice at time step t is denoted as $\mathcal{H} = \{d_1^t, d_2^t, \ldots, d_i^t, \ldots, d_N^t\}$, where where d_i^t represents the average packet drop rate caused by address shuffling in the slice i at time t. We define security vulnerability state at time t as $\mathcal{V} = \{v_1^t, v_2^t, \ldots, v_i^t \ldots, v_N^t\}$, where v_i^t represents the security vulnerability score in the slice i at time t. Note that the security vulnerability score is estimated by the degree of transmitting data packets to a same address. Taking all the above into consideration, the set of states at time step t is denoted as $\mathcal{S} = \{s_1^t, s_2^t, \ldots, s_i^t \ldots, s_N^t\}$. The state of the slice i at time step t, denoted by s_i^t, is as $s_i^t = [o_i^t, l_i^t, d_i^t, v_i^t]$.

2) ACTION SPACE

To satisfy the system security and performance requirements in each time step t, the slice controller determines the amount of bandwidth resource and the MTD shuffling interval of a slice and requests the determined resources to the central orchestrator. An action set consists of two sub-actions, one is for assigning network link bandwidth resource and the other for assigning the security resource (i.e., address shuffling interval). The range of link bandwidth assignment is in $[-\Delta\mathbb{B}, \Delta\mathbb{B}]$ as a continuous space, where $\Delta\mathbb{B}$ denotes the maximum value of bandwidth allocation change. We set $\Delta\mathbb{B} = \frac{\mathbb{C}}{N} \cdot p_{BW}$, where $\mathbb{C}$ is the capacity of the network link and N is the number of network slices in the network. In the experiment, we assume that the in-vehicle SDN environment is 1G Ethernet-based network with three network slices and 50 in-vehicle components. In this case, the range of link bandwidth allocation is in $[-50, 50]$ for $p_{BW} = 15\%$. The range of address shuffling interval is in $[\mathbb{T}_{min}, \mathbb{T}_{max}]$, where $\mathbb{T}_{min}$ and $\mathbb{T}_{max}$ denote the minimum and maximum shuffling interval, respectively. The values of $\mathbb{T}_{min}$ and $\mathbb{T}_{max}$ depend on network size and delay. In this work, we set $\mathbb{T}_{min} = 1$ and $\mathbb{T}_{max} = \mathbb{T}_{min} + 3$ empirically for our testbed. Then, the range of address shuffling interval is in $[1, 4]$. Note that if the values

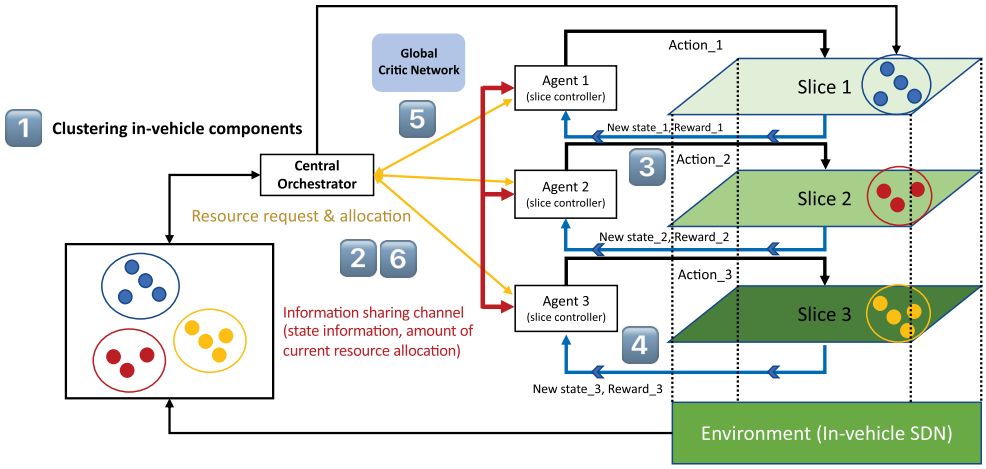


FIGURE 4. mDRL architecture for network slicing in in-vehicle SDN.

are very large, we can think of it as not applying the shuffling technique. The action taken at time step t by slice controller i, where $\mathcal{A}_{S_i^t}$ denotes the possible action set in the action space $\mathcal{A}$ under state S_i^t.

3) REWARD

The comprehensive reward of the network slice controller (i.e., an agent) is designed to reflect the following three aspects: (1) how satisfactorily the agent allocates bandwidth resources required for the slice; (2) the degree of security vulnerability exposed by using the same address in the data forwarding phase; and (3) the loss caused by the bandwidth allocation and address shuffling MTD.

We estimate the immediate reward associated with a given action based on the gain ($\mathcal{R}_G$) minus loss ($\mathcal{R}_L$). These gain and loss are denoted as $\mathcal{R}_G = \{r_1^g, r_2^g, \ldots, r_i^g, \ldots, r_N^g\}$, $\mathcal{R}_L = \{r_1^l, r_2^l, \ldots, r_i^l, \ldots, r_N^l\}$, where N is the number of slices (i.e., agents) in the in-vehicle network. We define total reward for each agent as follows: $\mathcal{R}_{total} = \mathcal{R}_G - \mathcal{R}_L$. The gain includes the benefit introduced in terms of the bandwidth resource allocation. As a result of the taken resource allocation action, the slice controller is rewarded if the amount of allocated bandwidth resource is larger than the required resource. Note that the maximum link bandwidth capacity is 1 Gbps. The gain, r_i^g, which represents the reward of the satisfaction rate of a slice i is estimated by:

$$r_i^g = e^{-(b_i^t - \hat{b}_i^t)^2}, \tag{3}$$

where b_i^t is the allocated bandwidth resource at time step t and $\hat{b}_i^t$ is the maximum data rate in the network at time step t. The loss, r_i^l, is calculated by:

$$r_i^l = \frac{v_i^t + p_i^t}{2}, \tag{4}$$

where v_i^t refers to the average vulnerability degree of components in the slice due to using the same IP address, and p_i^t is the packet loss ratio at the time slot t. The average vulnerability degree, v^t, is calculated by:

$$v_i^t = \frac{1}{k_i} \cdot \sum_{n=1}^{k_i} \frac{\hat{P}_n^t}{P_n^t}, \tag{5}$$

where k_i refers the total number of in-vehicle components in slice i, P_n^t is the total number of packets sent by the component n at time slot t, and $\hat{P}_n^t$ is the maximum number of packets sent by the component n with the same IP address at time slot t. Therefore, we define r_i^{total} for each agent as follows:

$$r_i^{total} = r_i^g - r_i^l. \tag{6}$$

Since both r_i^g and r_i^l are ranged in [0, 1] as a real number, r_i^{total} is ranged in [−1, 1] as a real number.

B. ANOMALY DETECTOR (AD) FOR FILTERING MANIPULATED OBSERVATIONS

To detect a manipulation attack in a state, we utilize a neural network called stacked long short term memory (LSTM) networks [34]. LSTM networks [35], a special form of recurrent neural networks (RNNs), have proven to be specifically useful for learning time series sequences involving long-term patterns due to their ability to maintain long-term history. In addition, LSTM networks are known to overcome the vanishing gradient problem caused by neural networks by applying a constant error flow through the internal states of special units (i.e., memory cells).

To detect anomalies in time series state observation information, a stated LSTM network is trained on normal data (e.g., no attacks) and used as a predictor over a number of time steps. [34] showed that stacking recurrent hidden layers in LSTM networks performs well in capturing the structure of time series data. LSTM units in the hidden layer are fully connected through recurrent connections, and each unit of the lower LSTM hidden layer is fully connected to each

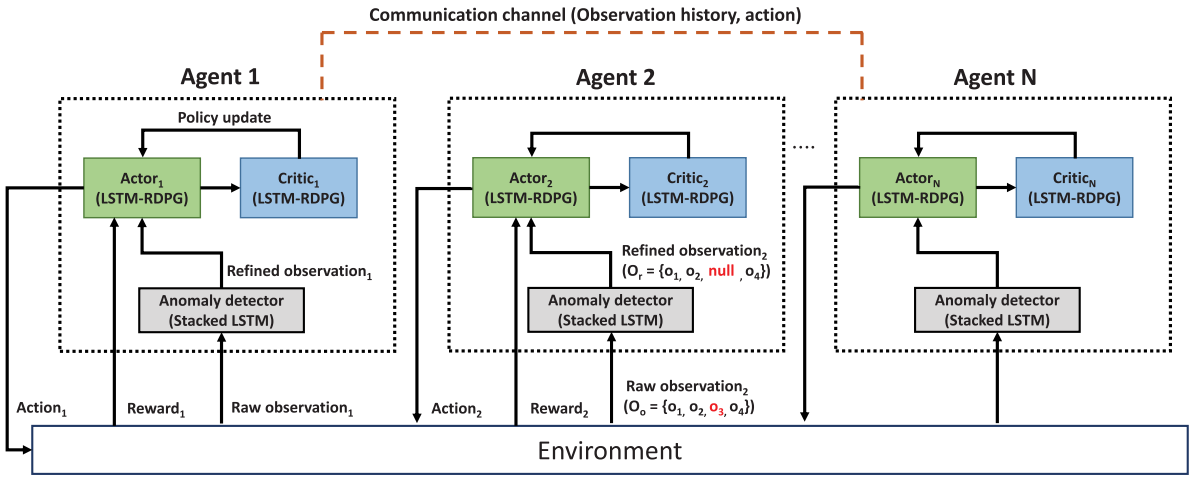


FIGURE 5. Proposed MARDPG-AD architecture.

unit of the LSTM hidden layer above it via a feed-forward connection. We take one unit in the input layer for each of the n dimensions of state information, and train LSTM networks to predict the next state information (i.e., the output layer generates the prediction value for the specified time step) from the previous three state information.

After the LSTM networks are trained to predict (i.e., after 600 episodes in Fig. 7(b)) the future observation information, the anomaly detector calculates the prediction error vector $E = Y - \hat{Y}$, where Y is the vector of true observation values and $\hat{Y}$ is the vector of prediction values. After calculating the prediction errors E, the errors are roughly fitted to a multivariate Gaussian distribution and the parameters of the distribution can be estimated. Then, the Mahalanobis distance is calculated by [36]:

$$D_M(\vec{e}_i) = \sqrt{(\vec{e}_i - \vec{m})^T S^{-1} (\vec{e}_i - \vec{m})}, \tag{7}$$

where S is a covariance matrix and $\vec{m}$ is a mean vector of the prediction errors E. The anomaly detector filters the manipulated observation in which the Mahalanobis distance is larger than a specified anomaly threshold, and nullifies the specific anomalous data from the raw observation information and sends the refined observation to the DRL agent as illustrated in Fig. 5.

C. PARTIALLY OBSERVABLE ENVIRONMENT

In MDP, we assume that the agent can always observe the complete state information of the environment and the next state of the environment only depends on the current state and the chosen action. However, in many real-world scenarios, assuming the next state of the MDP is only affected by the previous state may not be justified. For example, if the proposed anomaly detector filters the specific observation information and sends the refined state information to the agent, the agent can only see partial parts of the state information. In this case, a policy based on conventional MDP is not able to identify an optimal action to maximize the accumulated reward.

In order to deal with partial observability, a partially observable Markov decision process (POMDP) has been proposed. In POMDP, the environment information is considered not fully observable. It assumed that the DRL agent will observe system states with a probability at time t, denoted by p_{o_t}. Thus, an agent will augment its current observations using the observation and action information from previous steps. Therefore, an optimal policy needs a memory-based approximator which can store the entire history $h^t = (\mathbf{o}_1, \mathbf{a}_1, \mathbf{o}_2, \mathbf{a}_2, \ldots, \mathbf{a}_{t-1}, \mathbf{o}_t)$ because we cannot estimate a state precisely.

We utilize a memory-based policy gradient method called *recurrent deterministic policy gradient* (RDPG) [37]. Instead of using feed-forward neural networks, RDPG utilizes RNN-based memory, especially LSTM which has attracted in the RL area to predict some time-related data and learn preserving information about the past which is needed to solve the POMDP-based environment. RDPG extends the deterministic policy gradient (DPG) method of MDPs to deal with partially observable environments. Under partial observable environments, both the optimal policy (i.e., actor network) and the action-value function (i.e., critic network) are functions of the observation-action history, $h^t = (o_1, a_1, o_2, a_2, \ldots, a_{t-1}, o_t)$.

D. PROPOSED MULTI-AGENT RDPG (MARDPG)

In this work, we propose multi-agent recurrent deterministic policy gradient (MARDPG) algorithm, a combination of multi-agent model of the MADDPG and POMDP model of the RDPG, under an in-vehicle SDN environment with N agents (i.e., network slices) with policies parameterized by $\theta = \{\theta_1, \theta_2, \ldots, \theta_i, \ldots, \theta_N\}$, and let $\mu = \{\mu_1, \mu_2, \ldots, \mu_i, \ldots, \mu_N\}$ be the set of policies of all agents. Let $\mathbf{h}$ denote the entire history of all agents (i.e., $\mathbf{h} = (h_1, h_2, \ldots, h_i, \ldots, h_N)$). Then we can obtain the gradient

Algorithm 1 Multi-Agent Recurrent Deterministic Policy Gradient with Anomaly Detector for N Network Slices

// Initialization
Set critic network $Q_i^{\omega}(\mathbf{h}, \mathbf{a})$ and actor $\mu(h_i)$ with randomly generated weight ω_i and θ_i for all network slice agent $i \in N$
Set target networks $Q_i^{\omega'}$ and μ'_{θ_i} with weights $\omega'_i \leftarrow \omega_i$ and $\theta'_i \leftarrow \theta_i$
Set experience replay buffer $\mathcal{D}$
for episode $= 1$ to M **do**
  Initialize a random process $\mathcal{N}$ for exploration noise
  For each agent i, set initial history h_i^0
  for $t = 1$ to max-episode-length **do**
    For each agent i, receive refined observation o_i^t from the anomaly detector (AD_i)
    Construct new history $h_i^t \leftarrow h_i^{t-1}, a_i^{t-1}, o_i^t$
    Select action $a_i^t = \mu_{\theta_i}(h_i^t) + \mathcal{N}^t$
    Execute action $\mathbf{a^t} = (a_1^t, \ldots, a_N^t)$ and observe r, and new observation history $\mathbf{h}'$
    Store whole information $(\mathbf{h}, \mathbf{a^t}, \mathbf{r^t}, \mathbf{h}')$ for all agents in $\mathcal{D}$
    for agent $i = 1$ to N **do**
      Randomly sample a batch of T transitions $(\mathbf{h}, \mathbf{a^t}, \mathbf{r^t}, \mathbf{h}')$ from $\mathcal{D}$
      Set $y_i = r_i + \gamma Q^{\omega_i}(\mathbf{h}, a_1, \ldots, a_N)\,|_{a_j=\mu_j(h_j)}$
      Compute critic update by minimizing the loss: $\mathcal{L}(\theta_i) = \mathbb{E}_{\mathbf{h},a,r,\mathbf{h}'}[(Q_i^{\omega}(\mathbf{h}, a_1, \ldots, a_N) - y_i)^2]$
      Compute actor update: $\nabla_{\theta_i} J(\mu_i) = \mathbb{E}_{\mathbf{h},a\sim\mathcal{D}}[\nabla_{\theta_i}\mu_i(a_i \mid h_i)\nabla_{a_i} Q_i^{\omega}(\mathbf{h}, a_1, \ldots, a_N)]$
      Update actor and critic using Adam optimizer
    end for
  end for
end for

of the expected return for agent i based on the MADDPG algorithm by:

$$\nabla_{\theta_i} J(\mu_i) = \mathbb{E}_{\mathbf{h},a\sim\mathcal{D}}[\nabla_{\theta_i}\mu_i(a_i \mid h_i)\nabla_{a_i} Q_i^{\omega}(\mathbf{h}, a_1, \ldots, a_N)], \tag{8}$$

where ω is a recurrent neural network parameters, and a_i is the result of $\mu_{\theta_i}(h_i)$ (i.e., μ_i denote μ_{θ_i}). Experience replay buffer $\mathcal{D}$ contains the recording information of all agents' experience $(\mathbf{h}, \mathbf{h}', a_1, \ldots, a_N, r_1, \ldots, r_N)$. The centralized action-value function (i.e., critic network) Q_i^{ω} is used to minimize the following loss function in DL NNs:

$$\mathcal{L}(\theta_i) = \mathbb{E}_{\mathbf{h},a,r,\mathbf{h}'}[(Q_i^{\omega}(\mathbf{h}, a_1, \ldots, a_N) - y_i)^2], \tag{9}$$

$$y_i = r_i + \gamma Q_i^{\omega'}(\mathbf{h}', a'_1, \ldots, a'_N)\,|_{a'_j=\mu'_j(h_j)}, \tag{10}$$

where $Q^{\omega'}$ is the target network for the state-action value function, and μ'_{θ} is the target network for the actor in an agent. The basic motivation of the MADDPG is that if an agent knows the actions taken by all other agents, the environment remains stationary even if the policies of other agents change. By using the anomaly detector and RDPG concept together, the manipulated information detected by the anomaly detector is discarded, and the remaining information is used to derive observations of an agent in MARDPG as shown in Fig 5.

Our proposed MARDPG algorithm with an anomaly detector is described in Algorithm 1. At first, the actor and critic networks of each slice agent and the experience replay buffer are initialized with the random NN weights. Each agent distributively selects their action based on their actor network (i.e., policy) with random noise. Agents get rewards and next observation information after their own action. LSTM-based RDPG in critic and actor networks update their networks according to the observation and reward. In the centralized learning phase, each agent randomly samples a mini batch from $\mathcal{D}$ to train the critic and actor networks. and the critic networks in each agent estimate the value function by minimizing the loss function $\mathcal{L}(\theta_i)$. The actor networks are updated based on the loss function calculation. Finally, the target networks are updated based on the target network update rate τ_c. The computational complexity of our proposed Algorithm 1 is of $\mathcal{O}(N \cdot (\mathbb{H}_{\mathbb{A}}\mathbb{N}_{\mathbb{A}} + \mathbb{H}_{\mathbb{C}}\mathbb{N}_{\mathbb{C}}))$, where N denotes the number of agents, $\mathbb{H}_{\mathbb{A}}$ and $\mathbb{H}_{\mathbb{C}}$ denote the number of hidden layers of the actor and critic network, and $\mathbb{N}_{\mathbb{A}}$ and $\mathbb{N}_{\mathbb{C}}$ denote the number of neurons in the actor and critic network, respectively.

VI. EXPERIMENTAL SETTINGS

In this section, we describe the experimental setup for our testbed environment. We compare the performance of the proposed mDRL-based resource allocation method with other schemes in terms of various metrics. The network performance and security performance of the proposed scheme are evaluated by conducting the experiments measuring: (1) a resource utilization indicating the effective resource usage achieved by the proposed mDRL-based network slicing; (2) system security enhancement (i.e., degree of vulnerability) through the proposed MTD technique; and (3) QoS in terms of packet drop rate and throughput. Moreover, we investigate critical trade-offs between system performance (improving service availability) and security enhancement (minimizing system vulnerability).

A. TESTBED IMPLEMENTATION

To evaluate our proposed algorithm, we implemented a proof-of-concept prototype of an in-vehicle SDN. The in-vehicle SDN testbed consists of three DRL agents (i.e., slice controller), an ONOS SDN controller (i.e., central orchestrator), three Open vSwitches (OvSes), and 10 ECU-CAN nodes as shown in Figure 6. The ECU-CAN node is implemented with CAN-utils package in Linux to simulate a CAN environment, and the protocol adaptor based on TCP/IP sockets is implemented to convert CAN data frames to IP network packets. We utilize the CAN-SDN environment to generate in-vehicle network traffic and PyTorch to implement the mDRL model in the slice controllers.

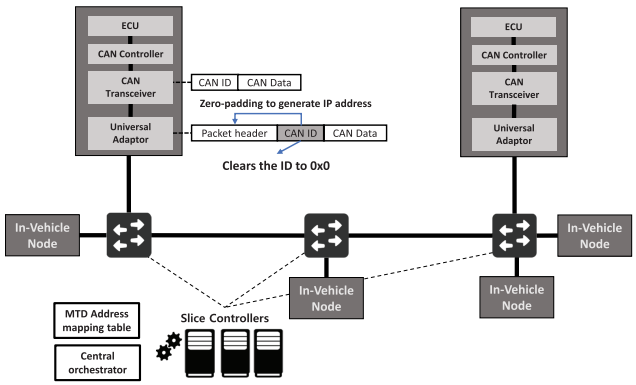


FIGURE 6. In-vehicle SDN testbed implementation.

The central orchestrator manages network resources for each slice by adjusting the size of slice buffers in the OvS switch, and security resources for each slice by adjusting address shuffling interval. The slice controller (i.e., DRL agent) decides forwarding policies in the network slice and changes the virtual addresses of each ECU component based on the assigned shuffling interval. When an IP packet from a universal adaptor is forwarded to the SDN switch, the slice controller overwrites the IP address with a random virtual IP address. This virtual address is used for forwarding the packets to the destination nodes. The virtual address in the packet header is converted to the real address at the switch right before the packet is forwarded to the destination ECU node. Then, the IP packet is reconstructed to the original CAN data frame in the universal adaptor.

In the experiment, the slice controller obtains the slice information (i.e., state information in Section V-A) from the SDN switches at every time step t using REST API. We assume that an attacker initiates state manipulation attacks after about 500 episodes, and in each time step, one of the observations from the environment has been contaminated with the attack success probability of 0.2.

B. METRICS

The following metrics are used to evaluate the performance of the proposed method and other compared counterparts:

- **Accumulated reward** ($\mathcal{R}$): This metric is the average value of the final accumulated rewards of each slice controller (i.e., agent) at the end of the simulation. We compare the three DRL algorithms in our work, as described in Section VI-C.
- **Bandwidth allocation success rate**: This metric measures the bandwidth resource allocated for a slice at time step t from the central orchestrator.
- **Degree of security vulnerability**: This metric measures the mean extent of security vulnerability in terms of the number of times data sent to the same address over the total number of data sent.
- **Packet loss rate**: This metric measures the packet loss rate of a slice at time step t. If the bandwidth resource required by the slice is not sufficiently allocated from the central orchestrator or the MTD shuffling interval is too short, packets in the slice are lost.

C. COMPARING SCHEMES

For the experimentation, we compare the performance of the following policy-based DRL algorithms which aim to find an optimal action directly from a policy network (i.e., actor) and the instant reward estimated based on the action taken:

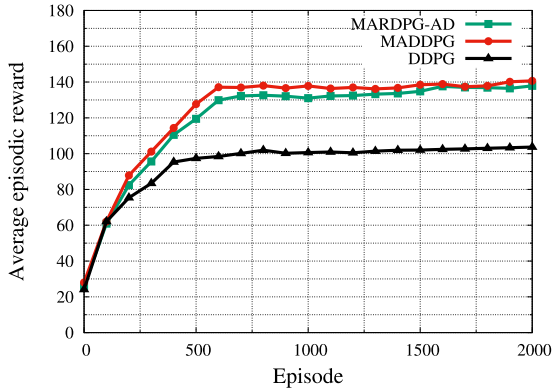
- **Multi-Agent Recurrent Deterministic Policy Gradient with Anomaly Detector (MARDPG-AD)**: This method is our proposed approach that extends the DDPG algorithm to deal with partially observable environments. RDPG uses off-policy to learn deterministic policies. It employs the RNNs, especially LSTM, for the agent to learn to preserve limited information about the past to deal with the imperfect observation. MARDPG extends RDPG by allowing multiple agents using RDPG to collaborate for enhanced learning; AD detects the anomalous state information from the in-vehicle environment and forwards the filtered observation information to the MARDPG agent.
- **Multi-Agent Deep Deterministic Policy Gradient (MADDPG)** [12]: This scheme extends DDPG to a multi-agent environment. The main idea is that if the actions of all the agents are well-known, then the policy space changes do not introduce the non-stationarity to the environment. This method deploys a central critic value function with deterministic policies. In MADDPG, each agent's actor network chooses the next action based on their current local observation information.
- **Deep Deterministic Policy Gradient (DDPG)** [7]: This scheme uses a single agent-based DRL that adopts a model-free, actor-critic algorithm to successfully deal with continuous action space problems. In our proposed system with three DRL agents, each agent does not exchange its action and reward information with other agents.

VII. PERFORMANCE EVALUATION

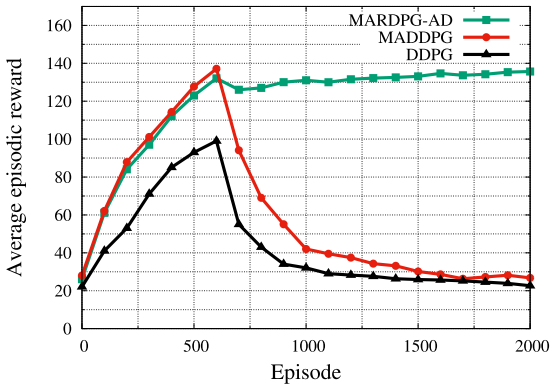
In this section, we demonstrate and discuss the experimental results that compare the performance of the three different DRL algorithms addressed in Section VI-C. Finally, we demonstrate the simulation results and discuss the overall insights observed from the results.

1) REWARD CONVERGENCE

Figure 7(a) shows how different mDRL and single-agent DRL algorithms can reach an accumulated reward as the agents learn from more episodes. We clearly observe that the proposed MARDPG-AD and the MADDPG outperform than

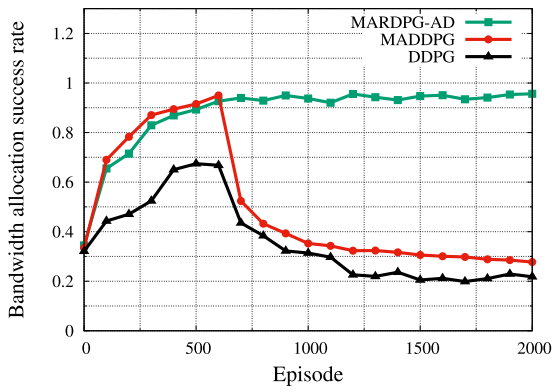


(a) Under non-anomalous environment.

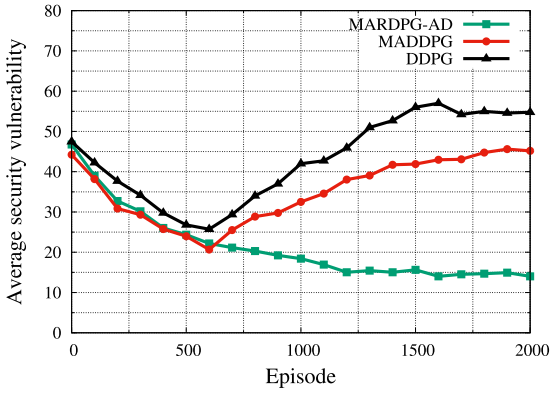


(b) Under adversarial attacks.

FIGURE 7. Accumulated reward with respect to the episodes.



(a) Result of the bandwidth resource allocation.



(b) Result of the average security vulnerability.

FIGURE 8. Success rate of the bandwidth and security resource allocation for the network slices.

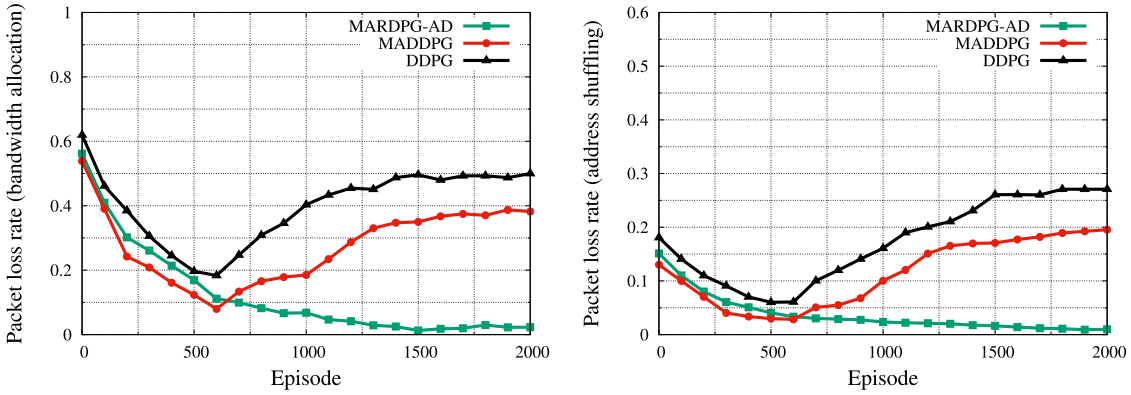
TABLE 3. Key design parameters and their default values.

Parameters	Default Value
Number of network slice N	3
Discount factor γ	0.99
Replay buffer $\mathcal{D}$	30,000
Batch size T	64
Target network update rate τ_c	0.001
Actor learning rate τ_a	0.0001
Simulation time steps per episode	500

single-agent DDPG method in terms of the average accumulated reward with respect to episodes in both non-anomalous and anomalous environments. In the absence of a state manipulation attack, the MADDPG performs slightly better than MARDPG-AD because the agent can obtain the optimal action based on the current state without having to remember the historical information. If there are no attacks in the environment, multi-agent policy gradient methods, such as MARDPG-AD and MADDPG, aim to identify an optimal action directly from a policy and corresponding reward estimated based on a taken action, which allows the agent to obtain more reward over the episodes than a single agent-based method (i.e., DDPG).

Figure 7(b) represents the average accumulated reward when the agents are under state manipulation attacks. We observe that the proposed MARDPG-AD agent controls the system stably by filtering out manipulated observation even in a situation under attack. On the other hand, the MADDPG and single-agent DDPG methods fail to find the optimal policies because of the manipulated state information. As the episodes progressed after the manipulation attacks are performed, the agents continued failing to allocate the bandwidth and security resources to each slice.

When we use a single-agent based DRL algorithm, as the slice controller is not aware of the actions of the other slice controllers, it does not know if the other agents are changing their policies or taking random actions to explore the environment. On the other hand, in our MARDPG-based proposed scheme, each agent keeps track of histories of their filtered observation information and rewards of other agents through the communication channel of the slice controllers. The communication informs the slice controller that the other agents have changed their policies or have taken actions. Therefore,



(a) Packet loss rate caused by bandwidth resource allocation.

(b) Packet loss rate caused by address shuffling.

FIGURE 9. Packet loss rate caused by bandwidth allocation and address shuffling.

in the proposed scheme, the slice controllers provide the capability to identify the optimized policies.

2) BANDWIDTH AND SECURITY RESOURCE MANAGEMENT

Fig. 8(a) shows the result of the average bandwidth allocation success rate of three network slices. At the initial episode, each slice controller is allocated an equal amount of the network bandwidth resources. We observe that each slice controller has successfully allocated the required bandwidth resources from the central orchestrator after about 500 episodes. Our result proves that the proposed mDRL-based scheme is capable of learning the optimal bandwidth resource management policy with respect to the ability to meet the resource requirements of a network slice, despite the presence of attackers' manipulation attacks after 500 episodes. On the other hand, the MADDPG shows good performance before the attack, but when an attack starts, resource allocation fails and the success rate rapidly decreases.

In Fig 8(b), we demonstrate how the mDRL agent's decisions achieve enhancing the system security level (i.e., average security vulnerability of three network slices). Our proposed method has successfully found the optimal address shuffling interval for each network slice. However, when an attack starts, MADDPG and DDPG fail to find the optimal address shuffling interval due to the high packet loss rate.

3) PACKET LOSS RATE CAUSED BY BANDWIDTH ALLOCATION AND MTD

In Fig. 9, we demonstrate how the mDRL agent's decisions achieve enhancing the service availability goal of a slice in terms of the packet loss rate. Fig. 9(a) shows that the packet loss rate due to the bandwidth allocation from the slice controller. Fig. 9(b) shows that the packet loss rate caused by address shuffling. In both experiments, our proposed MARDPG-AD algorithm increases the service availability of the system by gradually reducing the packet loss rate even under the manipulation attacks. On the other hand, the two comparing algorithms fail to find the optimal bandwidth allocation and address shuffling intervals due to the manipulation attacks.

VIII. CONCLUSION

We developed a deep reinforcement learning (DRL)-based resource allocation and MTD deployment framework, named DESOLATER (Drl-based rESOurce aLlocation And mTd dEployment fRamework). The DESOLATER aimed to effectively identify an optimal amount of link bandwidth to each network slice which has a DRL agent running and an optimal interval to trigger IP shuffling-based MTD based on the critical tradeoff between security and performance.

We obtained the following **key findings** from our study and identified the corresponding future work directions:

- A multi-agent based DRL approach deployed in network slicing for resource allocation and MTD triggering decisions is more effective than a single agent-based DRL counterpart. The reason is because exchanging information between agents can improve the synchronization of views and policies, which leads to each DRL agent making more effective decisions under reduced uncertainty.
- For the resource allocation optimization, the convergence of optimal actions meeting the required demand of each slide in bandwidth has reached after 600 episodes. In future work, we will shorten the learning curves by introducing more aggressive approaches for an agent to learn faster.
- For the optimal MTD operation minimizing security vulnerability and packet loss, a shorter MTD shuffling interval has been chosen to minimize security vulnerability. In addition, when the packet loss rate increases, a longer MTD interval is chosen by DRL agents. Currently, the security vulnerability is estimated by the extent of packets using the same address, which is significantly affected by the packet arrival rates. In future work, we will introduce new security metrics that can

capture the actual security breach by attackers when the MTD triggering interval is adjusted.

ACKNOWLEDGMENT

The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of CCDC ITC-PAC, CCDC-ARL, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation hereon.

REFERENCES

[1] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, "Toward proactive, adaptive defense: A survey on moving target defense," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 709–745, 1st Quart., 2020.

[2] K. Halba and C. Mahmoudi, "In-vehicle software defined networking: An enabler for data interoperability," in *Proc. 2nd Int. Conf. Inf. Syst. Data Mining*, Apr. 2018, pp. 93–97.

[3] S. Yoon, T. Ha, S. Kim, and H. Lim, "Scalable traffic sampling using centrality measure on software-defined networks," *IEEE Commun. Mag.*, vol. 55, no. 7, pp. 43–49, Jul. 2017.

[4] B. Yi, X. Wang, K. Li, S. K. Das, and M. Huang, "A comprehensive survey of network function virtualization," *Comput. Netw.*, vol. 133, pp. 212–262, Mar. 2018.

[5] Y. Li and M. Chen, "Software-defined network function virtualization: A survey," *IEEE Access*, vol. 3, pp. 2542–2553, 2015.

[6] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2014, pp. 387–395.

[7] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016, pp. 1–14.

[8] J. Zhang, Y. Pan, H. Yang, and Y. Fang, "Scalable deep multi-agent reinforcement learning via observation embedding and parameter noise," *IEEE Access*, vol. 7, pp. 54615–54622, 2019.

[9] Y. Wang, H. Liu, W. Zheng, Y. Xia, Y. Li, P. Chen, K. Guo, and H. Xie, "Multi-objective workflow scheduling with deep-Q-network-based multi-agent reinforcement learning," *IEEE Access*, vol. 7, pp. 39974–39982, 2019.

[10] Z. Sui, Z. Pu, J. Yi, and X. Tan, "Path planning of multiagent constrained formation through deep reinforcement learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–8.

[11] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.* Cham, Switzerland: Springer, 2017, pp. 66–83.

[12] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 6382–6393.

[13] D. C. MacFarland and C. A. Shue, "The SDN shuffle: Creating a moving-target defense using host-based software-defined networking," in *Proc. ACM Workshop Moving Target Defense (MTD)*, 2015, pp. 37–41.

[14] S. Achleitner, T. F. La Porta, P. McDaniel, S. Sugrim, S. V. Krishnamurthy, and R. Chadha, "Deceiving network reconnaissance using SDN-based virtual topologies," *IEEE Trans. Netw. Service Manage.*, vol. 14, no. 4, pp. 1098–1112, Dec. 2017.

[15] J. B. Hong, S. Yoon, H. Lim, and D. S. Kim, "Optimal network reconfiguration for software defined networks using shuffle-based online MTD," in *Proc. IEEE 36th Symp. Reliable Distrib. Syst. (SRDS)*, Sep. 2017, pp. 234–243.

[16] J. H. Jafarian, E. Al-Shaer, and Q. Duan, "OpenFlow random host mutation: Transparent moving target defense using software defined networking," in *Proc. ACM SIGCOMM Workshop Hot Topics Softw. Defined Netw. (HotSDN)*, 2012, pp. 127–132.

[17] A. Chowdhary, S. Pisharody, and D. Huang, "SDN based scalable MTD solution in cloud network," in *Proc. ACM Workshop Moving Target Defense (MTD)*, Oct. 2016, pp. 27–36.

[18] P. Kampanakis, H. Perros, and T. Beyene, "SDN-based solutions for moving target defense network protection," in *Proc. IEEE Int. Symp. World Wireless, Mobile Multimedia Netw.*, Jun. 2014, pp. 1–6.

[19] E. M. Ghourab, E. Samir, M. Azab, and M. Eltoweissy, "Diversity-based moving-target defense for secure wireless vehicular communications," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2018, pp. 287–292.

[20] S. Vikram, C. Yang, and G. Gu, "NOMAD: Towards non-intrusive moving-target defense against Web bots," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Oct. 2013, pp. 55–63.

[21] M. Green, D. C. MacFarland, D. R. Smestad, and C. A. Shue, "Characterizing network-based moving target defenses," in *Proc. 2nd ACM Workshop Moving Target Defense*, Oct. 2015, pp. 31–35.

[22] E. M. Ghourab, E. Samir, M. Azab, and M. Eltoweissy, "Trustworthy vehicular communication employing multidimensional diversification for moving-target defense," *J. Cyber Secur. Mobility*, vol. 8, no. 2, pp. 133–164, 2019.

[23] M. Albanese, A. De Benedictis, S. Jajodia, and K. Sun, "A moving target defense mechanism for MANETs based on identity virtualization," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Oct. 2013, pp. 278–286.

[24] S. Woo, D. Moon, T.-Y. Youn, Y. Lee, and Y. Kim, "CAN ID shuffling technique (CIST): Moving target defense strategy for protecting in-vehicle CAN," *IEEE Access*, vol. 7, pp. 15521–15536, 2019.

[25] G. Sun, K. Xiong, G. O. Boateng, G. Liu, and W. Jiang, "Resource slicing and customization in RAN with dueling deep Q-network," *J. Netw. Comput. Appl.*, vol. 157, May 2020, Art. no. 102573.

[26] H. Wang, Y. Wu, G. Min, J. Xu, and P. Tang, "Data-driven dynamic resource scheduling for network slicing: A deep reinforcement learning approach," *Inf. Sci.*, vol. 498, pp. 106–116, Sep. 2019.

[27] G. Sun, K. Xiong, G. O. Boateng, D. Ayepah-Mensah, G. Liu, and W. Jiang, "Autonomous resource provisioning and resource customization for mixed traffics in virtualized radio access network," *IEEE Syst. J.*, vol. 13, no. 3, pp. 2454–2465, Sep. 2019.

[28] G. Sun, Z. T. Gebrekidan, G. O. Boateng, D. Ayepah-Mensah, and W. Jiang, "Dynamic reservation and deep reinforcement learning based autonomous resource slicing for virtualized radio access networks," *IEEE Access*, vol. 7, pp. 45758–45772, 2019.

[29] S. Yoon, J.-H. Cho, D. S. Kim, T. J. Moore, F. Nelson, and H. Lim, "Poster: Address shuffling based moving target defense for in-vehicle software-defined networks," in *Proc. ACM Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2019, pp. 1–3.

[30] S. Yoon, J.-H. Cho, D. S. Kim, T. J. Moore, F. F. Nelson, H. Lim, N. Leslie, and C. Kamhoua, "Moving target defense for in-vehicle software-defined networking: IP shuffling in network slicing with multiagent deep reinforcement learning," *Proc. SPIE*, vol. 11413, Apr. 2020, Art. no. 114131U.

[31] P. Fussey and G. Parisis, "Poster: An in-vehicle software defined network architecture for connected and automated vehicles," in *Proc. 2nd ACM Int. Workshop Smart, Auton., Connected Veh. Syst. Services*, Oct. 2017, pp. 73–74.

[32] T. Taleb, I. Afolabi, K. Samdanis, and F. Z. Yousaf, "On multi-domain network slicing orchestration architecture and federated resource control," *IEEE Netw.*, vol. 33, no. 5, pp. 242–252, Sep. 2019.

[33] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 2429–2453, 3rd Quart., 2018.

[34] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," in *Proceedings*, vol. 89. Ottignies-Louvain-la-Neuve, Belgium: Presses Universitaires de Louvain, 2015, pp. 89–94.

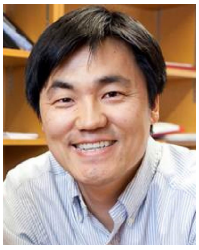
[35] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[36] R. De Maesschalck, D. Jouan-Rimbaud, and D. L. Massart, "The Mahalanobis distance," *Chemometrics Intell. Lab. Syst.*, vol. 50, no. 1, pp. 1–18, 2000.

[37] N. Heess, J. J Hunt, T. P Lillicrap, and D. Silver, "Memory-based control with recurrent neural networks," 2015, *arXiv:1512.04455*. [Online]. Available: http://arxiv.org/abs/1512.04455

SEUNGHYUN YOON (Graduate Student Member, IEEE) received the B.S. degree in computer science from Handong Global University, South Korea, in 2016, and the Ph.D. degree in electrical engineering and computer science from the Gwangju Institute of Science and Technology (GIST), South Korea, in 2021. From August 2019 to February 2020, he was a Visiting Scholar with the Department of Computer Science, Virginia Tech. He is currently a Senior Researcher with the Korea National Engineering Technology Center, Korea Institute of Industrial Technology (KITECH). His research interests include software-defined networking (SDN), network resource allocation and optimization, deep reinforcement learning, cybersecurity for various network domains, and moving target defense (MTD).

JIN-HEE CHO (Senior Member, IEEE) received the M.S. and Ph.D. degrees in computer science from Virginia Tech, in 2004 and 2008, respectively. She has been an Associate Professor with the Department of Computer Science, Virginia Tech, since August 2018, and the Director of the Trustworthy Cyberspace Lab. Prior to joining the Virginia Tech, she has been working as a Computer Scientist with the U.S. Army Research Laboratory (USARL), Adelphi, MD, USA, since 2009. She has published over 120 peer-reviewed technical articles in leading journals and conferences in the areas of trust management, cybersecurity, metrics and measurements, network performance analysis, resource allocation, agent-based modeling, uncertainty reasoning and analysis, information fusion/credibility, and social network analysis. She is also a member of ACM. She received the best paper awards from IEEE TrustCom'2009, BRIMS'2013, IEEE GLOBECOM'2017, 2017 ARL's Publication Award, and IEEE CogSima 2018. She is a winner of the 2015 IEEE Communications Society William R. Bennett Prize in the field of communications networking. In 2016, she was selected for the 2013 Presidential Early Career Award for Scientists and Engineers (PECASE), which is the highest honor, bestowed by the U.S. government on outstanding scientists and engineers in the early stages of their independent research careers.



DONG SEONG KIM (Senior Member, IEEE) received the Ph.D. degree from Korea Aerospace University, in February 2008. He was a Visiting Scholar with the University of Maryland, College Park, MD, USA, in 2007. From June 2008 to July 2011, he was a Postdoctoral Researcher with Duke University. He was a Senior Lecturer/Lecturer of Cybersecurity with the University of Canterbury, from August 2011 to December 2018. He has been an Associate Professor of Cybersecurity with The University of Queensland, Australia, since January 2019. His research interests include automated cybersecurity modeling and analysis for the Internet of Things, Cloud computing, and moving target defense. He was the General Co-Chair of ACISP2019 and the General Chair of IEEE PRDC 2017. He has served as a Program Co-Chair for IEEE TrustCom2019, IEEE ICIOT2019, ATIS2017, GraMsec2015, IEEE DASC2015, and a program committee member of international conferences, including IFIP/IEEE DSN, ISSRE, SRDS, ICC CISS, respectively.

TERRENCE J. MOORE (Member, IEEE) received the B.S. and M.A. degrees in mathematics from American University, in 1998 and 2000, respectively, and the Ph.D. degree in mathematics from the University of Maryland, College Park, MD, USA, in 2010. He is currently a Researcher with the Network Science Division, U.S. Army Research Laboratory. His research interests include sampling theory, constrained statistical inference, stochastic optimization, network security, geometric and topological applications in networks, and network science.

FREDERICA FREE-NELSON is currently a Researcher and a Program Lead with the U.S. Army Research Laboratory (ARL), Adelphi, MD, USA, where she leads research on machine learning and intrusion detection methods and techniques to promote cyber resilience and foster research on autonomous active cyber defense. She manages and negotiates the Research and Project Agreements for ARL between the Network Security Branch and Academia or International Organizations. She is also the lead for the Robust low-level cyber-attack resilience for Military Defense (ROLLCAGE) program working in collaboration with the Army Tank Automotive Research, Development and Engineering Center (TARDEC), the Office of Naval Research (ONR), and the Air force Research Laboratory (AFRL) to build a cohesive in-vehicular resilient system for defense against sophisticated enemy malware that strives to blend in with normal system activities. She has over 20 years' combined experience in cybersecurity research, software engineering, and program management within the DoD and other federal services to include the Federal Bureau of Investigation (FBI) and the Department of Justice. She has expertise in leading projects to success from conception to execution and delivery/transfer. She also serves as the Chairperson for the International Science Technology (IST-163) Panel–NATO Science and Technology Organization (STO) on the topic of deep machine learning for military cyber defense. She is also a participant in the Army Education Outreach Program as an Ambassador and a Virtual Judge for the eCybermission program.

HYUK LIM (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from the School of Electrical Engineering and Computer Science, Seoul National University, Seoul, Republic of Korea, in 1996, 1998, and 2003, respectively. From 2003 to 2006, he was a Postdoctoral Research Associate with the Department of Computer Science, University of Illinois at Urbana-Champaign, Champaign, IL, USA. He is currently a Full Professor with the AI Graduate School and jointly with the School of Electrical Engineering and Computer Science, Gwangju Institute of Science and Technology (GIST), Gwangju, South Korea. His research interests include wired and wireless networks, cyber-security, and artificial intelligence.